

Нужно реализовать **минимально рабочий сервис** с 3 уровнями профилей:

- **Profile+**
- **Profile**
- **Profile min**

Важно: это должно быть реализовано **под десктоп / компьютерную версию, не под мобильную**.

Мы хотим увидеть **полноценные экраны**, а не мобильную адаптацию или упрощенный интерфейс “на телефон”.

Логика профилей

1. Profile+

Поля:

- ID Profile+
- комиссия от системы
- срок оплаты
- счетчики:
 - общая сумма
 - оплачено
 - остаток
- количество заказов
- количество суб-аккаунтов ниже уровнем
- кнопка / действие «Открыть»

При открытии Profile+ должны быть видны:

- данные профиля
- его заказы
- список вложенных **Profile**
- возможность провалиться в каждый вложенный Profile

2. Profile

Поля:

- ID Profile
- ID Profile+

(если принадлежит аккаунту уровнем выше, а не был создан системой)

- комиссия от ID Profile+

(если Profile создан под Profile+, эта связь должна учитываться)

- срок оплаты
- счетчики:
 - общая сумма

- оплачено
 - остаток
- количество заказов
- количество суб-аккаунтов ниже уровнем
- кнопка / действие «Открыть»

При открытии Profile должны быть видны:

- данные профиля
- его заказы
- список вложенных **Profile min**
- возможность провалиться в каждый вложенный Profile min

3. Profile min

Поля:

- ID
- ID Profile+
- ID Profile

(то есть должна отображаться цепочка принадлежности)

- если Profile min создан системой, связь можно не указывать
- комиссия
- срок оплаты
- счетчики:
 - общая сумма
 - оплачено
 - остаток
- количество заказов
- кнопка / действие «Открыть»

При открытии Profile min должны быть видны:

- данные профиля
- его заказы

Заказы

У каждого профиля могут быть свои заказы.

Поля заказа:

- номер
- статус:
 - оплачено
 - не оплачено
 - частично оплачено
- сумма

- оплачено
- остаток

Заказы должны отображаться **в виде таблицы**.

В таблице должна быть возможность **внести сумму оплаты прямо в строке заказа**.

После внесения оплаты должны автоматически пересчитываться:

- оплачено
- остаток
- статус заказа

Логика статуса:

- если $\text{оплачено} = 0 \rightarrow$ **не оплачено**
- если $\text{оплачено} > 0$, но меньше суммы $\rightarrow$ **частично оплачено**
- если $\text{оплачено} = \text{сумме} \rightarrow$ **оплачено**

Что нужно сделать

Нужно реализовать:

- backend-логику
- базовый frontend
- хранение и связи между уровнями профилей
- хранение заказов
- открытие карточек профилей
- переходы по иерархии вниз
- таблицу заказов
- внесение оплаты по заказу
- автоматический пересчет счетчиков

Для каждого профиля должны считаться:

- общая сумма по заказам
- оплачено
- остаток
- количество заказов
- количество суб-аккаунтов ниже уровнем

По frontend

Дизайн не является приоритетом.

Нужен **минимальный, но рабочий интерфейс**, чтобы можно было руками проверить логику.

При этом интерфейс должен быть сделан именно **как desktop-версия**:

- с полноценными экранами
 - с нормальной структурой страниц
 - с удобным размещением таблиц, карточек и навигации
 - без ориентации на мобильный формат
-

По стеку

Для нас **не принципиально, на чем это будет сделано**.

Можно использовать любой адекватный стек.

Также можно использовать **AI / нейросети / генераторы кода / UI-библиотеки**.

Это не запрещено. Наоборот, нам важен **рабочий результат**, а не то, насколько “вручную” все написано.

Что нужно прислать по итогу

1. Ссылку
 2. Инструкцию по запуску
 3. Краткое описание структуры проекта
 4. Краткое описание модели данных и логики связей
-

Что нам важно увидеть

Нам важно посмотреть:

- как вы разбираете бизнес-логику
- как строите связи между сущностями
- как организуете backend
- как собираете базовый frontend
- как принимаете решения по структуре
- насколько быстро и практично доводите задачу до рабочего состояния

Главный критерий:

решение должно работать, быть логичным и проверяемым руками.

Ограничение по времени

Просим подойти к заданию **как к быстрому рабочему прототипу**, а не как к полноценной продуктовой разработке.

Ожидаемое время выполнения — не более 5 часов.

Нам не нужно, чтобы вы “вылизывали” интерфейс, архитектуру или детали реализации.

Главное — показать:

- логику решения
- понимание структуры сущностей и связей
- умение быстро собрать рабочий backend + базовый frontend
- способность уложиться в ограничение по времени и при этом выдать понятный результат

Мы осознанно смотрим на результат с учетом лимита в 5 часов.

Поэтому для нас важнее **ход мысли, логика, структура и работоспособность**, чем полировка и идеальность.